Day 1 — Orientation and Setup

ECON 300 · Applied Statistics for Economists

Ari Kamau

2026-09-10

MASTER DECK. Every "You Try" slide is followed immediately by its solution slide, so you can present straight through and reveal the answer by advancing one slide.

Press S for this speaker-notes view, B for the chalkboard, ESC for the overview.

TIMING FOR THE CODE HOUR: 0-15 Positron, R, package install running in background 15-25 Python and uv 25-30 Quarto and Git 30-35 Accounts (news, Census) 35-50 The three You Try blocks 50-60 Setup verification assignment, questions

Start install.packages() at minute 2 and let it run while you talk. It is the longest single wait of the morning.

This morning

Hour 1 · Theory

- What this course is, and what it is not
- Why statistics, and why *applied*
- Statistics vs. data science vs. machine learning
- The shape of the next ten weeks
- Deadlines that do not move

Hour 2 · Code

- Positron
- R 4.4 and the tidyverse
- Python 3.12 and `uv`
- Quarto
- Git
- Two accounts: WSJ and the Census API

Reading for today: Cohen, *Modern Statistics*, Chapter 1.

Hour 1 — What this course is for

The one-sentence version

You will learn to take a question about the world, find data that bear on it, and produce an answer that someone else can reproduce.

. . .

Every word in that sentence is doing work.

- **a question about the world** — not a textbook exercise
- **data that bear on it** — usually messy, usually not designed for your question
- **an answer** — with an honest account of its uncertainty
- **someone else can reproduce** — code, not clicking

What this course is not

- It is not a proofs course. We derive when the derivation teaches you something, and we simulate when it does not.
- It is not a software tutorial. You will learn two languages, but the languages are the means.
- It is not a course where you watch me type.

. . .

> This is a class of one. There is no back row. If you are lost and silent, nothing happens; if you are lost and say so, we stop and fix it.

Why learn statistics at all?

Cohen opens Chapter 1 with a claim worth taking seriously: statistics is how you avoid fooling yourself.

- Data are noisy. Patterns appear in noise. Humans are extremely good at seeing patterns that are not there.
- Statistics is the discipline of asking *how surprised should I be by this pattern, if nothing were going on?*

- Everything else — the tests, the intervals, the models — is machinery built to answer that one question in specific settings.

. . .

If you remember nothing else from today: **the null hypothesis is a claim about what noise alone would produce.**

Say "what noise alone would produce" every single time for the first three weeks. The textbook phrasing ("no effect") invites the wrong intuition that H0 is a claim you believe.

Statistics, data science, machine learning

	Asks	Optimizes for	Typical output
Statistics	Is this real?	Correct inference	An estimate and its uncertainty
Data science	What is in here?	Insight and communication	A finding, a dashboard
Machine learning	What happens next?	Predictive accuracy	A model that generalizes

. . .

They overlap heavily and the borders are marketing as much as substance. But the *question* differs, and the question determines what counts as a good answer.

If this lands abstractly, use the concrete case: PREDICTING which loans default (ML) versus TESTING whether a lender's approval rate differs by neighborhood (statistics). Same data, different question, different standard of evidence.

Economics sits awkwardly across all three

- We usually want a **causal** claim: did the policy do that?
- We almost never get to run the experiment.
- So we spend enormous effort on identification — arguing that a comparison is *as if* random.
- That argument is not statistical. It is economic. The statistics comes after.

. . .

> This course builds the statistical foundation. The identification argument is what your econometrics sequence adds on top of it.

Do not go deep today. Plant the flag and move on. It returns in the markets block on 11/3.

An opener, if we have time

> A bank's data show that applicants from one zip code are approved at 62%, and from another at 81%.

1. What would make that difference **uninteresting**?
2. What would make it **interesting**?
3. What data would you need to tell those apart?
4. Suppose the gap survives every control you have. Have you shown discrimination?

Expected answers: 1. Different applicant pools - income, credit depth, loan size. 2. Same observable risk profile, different outcome. 3. Applicant-level covariates, not zip-code averages. 4. No. You have shown an unexplained gap. The remaining step is an argument about what is unobserved.

This is the whole course in four questions. It also previews the cleaning block: most of the work is getting to applicant-level data you trust.

The shape of the term

Weeks	Block	The question it answers
9/15 – 9/24	Descriptives and distributions	What do these data look like?
9/29 – 10/8	Probability	What would noise alone produce?
10/13 – 10/22	Data cleaning	Can I trust these numbers?
10/27 – 10/29	Bayes	What should I believe, given what I knew before?
11/3 – 11/17	Prediction and financial markets	Does any of this survive contact with a market?
11/19	**Final presentation**	Can you defend it?

The deadlines that do not move

- **Mon 9/14, 11:59 PM** — setup verification (pass/fail, gates everything)
- **Tue 9/15, 6:15 AM** — news subscription active
- **Every Monday, 11:59 PM** — problem set, nine of them
- **Thu 10/29, 6:15 AM** — final project proposal
- **Wed 11/18, 11:59 PM** — final paper and repository
- **Thu 11/19, 6:15 AM** — you present

. . .

Late problem sets lose 10% per day. Not because lateness is a sin, but because the set is scaffolding for the next session.

How to actually succeed here

1. **Do the reading before class.** The theory hour goes badly otherwise.
2. **Type during the code hour.** Watching is not learning.
3. **Struggle on the problem set before asking.** Twenty minutes of being stuck is worth two hours of being shown.
4. **Break things on purpose.** Delete a line and see what error you get. Errors are the cheapest information available.

. . .

> Keep a file called `questions.md` in your course repo. Every time something confuses you, write it down. Bring it Tuesday.

Hour 2 — Setup

What we are installing, and why

Tool	What it is	Why this one
Positron	The IDE	Speaks R and Python natively, in one window
R 4.4	Language	Best statistical ecosystem, unmatched for data manipulation
Python 3.12	Language	Best for ML, APIs, general programming

Tool	What it is	Why this one
Quarto	Document engine	Runs both languages, produces papers *and* these slides
Git	Version control	Your project must run from a clean clone

. . .

Positron is Posit's newer IDE. If you have used RStudio, everything you know transfers.

Why two languages

- A t-test is a t-test. Seeing the same idea in two syntaxes separates the concept from the incantation.
- R is better at the things statisticians do: data frames, factors, formulas, plotting.
- Python is better at the things engineers do: APIs, pipelines, scikit-learn, anything touching production.
- Economists in 2026 are expected to read both.

. . .

Our convention this term: R is the spine. Python appears whenever it does the job better, and takes over entirely for the cleaning and markets blocks.

Step 1 — Positron

1. Download from positron.posit.co
2. Install and open it
3. Install the **R** and **Python** extensions from the Extensions pane

. . .

> Positron will not find R or Python until they are installed. Do steps 2 and 3 first if it complains.

Step 2 — R and the tidyverse

Install R 4.4+ from cran.r-project.org, then in the R console:

```
install.packages(c(
  "tidyverse", # dplyr, ggplot2, tidyr, readr, and friends
  "renv", # per-project package management
  "knitr",
  "rmarkdown",
  "tidycensus", # Census and ACS data - we start using this next week
  "janitor", # cleaning helpers
  "scales", # axis formatting
  "reticulate" # lets one Quarto file run R and Python chunks together
))
```

. . .

This takes a few minutes. Start it, then read the next slide while it runs.

Step 3 — Python and `uv`

`uv` is a fast package and environment manager. One tool instead of `pip`, `venv`, and `pyenv`.

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh

# then, inside your course folder:
uv init econ300
cd econ300
uv add numpy pandas matplotlib scipy statsmodels scikit-learn jupyter
```

. . .

> `uv` writes a lockfile. That is what makes your project reproducible on a machine that is not yours — which is exactly what the final project is graded on.

Step 4 — Quarto

Download from quarto.org.

Quarto is what turns a `.qmd` file into HTML, PDF, Word — or a slide deck like this one.

. . .

> These slides *are* a Quarto document. By the end of next Tuesday you will be able to make one.

Step 5 — Git

```
git --version          # macOS will offer to install it if missing

git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

Then create a repository for this course:

```
mkdir econ300 && cd econ300
git init
echo "# ECON 300 - Applied Statistics" > README.md
git add README.md
git commit -m "First commit"
```

Two accounts to set up

News subscription

Required, active by **Tue 9/15, 6:15 AM**.

- WSJ student rate, or Howard Libraries institutional access
- FT, Bloomberg, or The Economist also acceptable

Every Tuesday we open with an article. You will not know which one in advance.

Census API key

Free, takes two minutes: api.census.gov/data/key_signup.html

```
library(tidycensus)
census_api_key("YOUR_KEY_HERE", install = TRUE)
```

We start pulling real economic indicators next week.

If asked why a WSJ subscription belongs in a statistics class: because the hardest part of the final project is not the method, it is finding a question worth asking. That reflex is trained by reading, not by lectures.

You Try — is R alive?

In the Positron console, run each of these and predict the answer **before** you press enter.

```
2 + 2

sqrt(144)

c(1, 2, 3) * 10

mean(c(4, 8, 15, 16, 23, 42))
```

. . .

Now break it on purpose:

```
mean(c(4, 8, "fifteen", 16))
```

What error do you get, and what is R telling you about types?

Solution — is R alive?

```
2 + 2
#> [1] 4

sqrt(144)
#> [1] 12
```

```r
c(1, 2, 3) * 10
#> [1] 10 20 30

mean(c(4, 8, 15, 16, 23, 42))
#> [1] 18
```

Solution — the deliberate error

```r
mean(c(4, 8, "fifteen", 16))
#> Warning in mean.default(c(4, 8, "fifteen", 16)): argument is not numeric or
#> logical: returning NA
#> [1] NA
```

. . .

```r
c(1, 2, "three") # look at what actually came back
#> [1] "1"     "2"     "three"
```

> `c()` cannot hold mixed types, so it **silently coerces everything to character**. `mean()` then returns `NA` with a warning rather than an error. The lesson is not "don't mix types" — it is that **R will quietly change your data to make an operation possible**.

This is a preview of the cleaning block. Silent coercion is how a -999 sentinel survives into a regression.

You Try — is Python alive?

In the Python console:

```python
2 + 2

import numpy as np
np.sqrt(144)

np.array([1, 2, 3]) * 10

np.mean([4, 8, 15, 16, 23, 42])
```

. . .

Then try this, and explain why it is different:

```
[1, 2, 3] * 10
```

Solution — is Python alive?

```
2 + 2
#> 4

import numpy as np
np.sqrt(144)
#> np.float64(12.0)

np.array([1, 2, 3]) * 10
#> array([10, 20, 30])

[1, 2, 3] * 10        # plain Python list - NOT elementwise
#> [1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, ...]  repeats the list ten times
```

. . .

> **R vectorizes by default; Python does not.** `numpy` adds that behavior; plain Python lists repeat instead of multiplying. That single difference explains most of the syntax gap you will feel for the first two weeks.

You Try — your first Quarto document

Create `setup-check.qmd` in your course folder. Fill in the blanks, then press **Render**.

````
---
title: "____"
author: "____"
format: html
---

## R works

```{r}
````
````

````
# Print the version of R you are running.
# Hint: R.version.string
````

````
---
```

## Python works

```{python}
# Print the version of Python you are running.
# Hint: import sys, then sys.version
```
---
````

Solution — your first Quarto document

````
---
title: "Setup Check"
author: "Your Name"
format: html
---

## R works

```{r}
R.version.string
```

## Python works

```{python}
import sys
print(sys.version)
```
````

If the Python chunk errors with “Python engine not found”, the fix is install.packages(“reticulate”) then restart Positron. Budget five minutes for hitting this.
````

Your assignment for Sunday night

> ❗ Setup verification — due Monday 9/14, 11:59 PM · pass/fail
>
> Render a `.qmd` to HTML that proves all five of these work, and push it to your repo.

1. An R chunk printing `R.version.string`
2. A Python chunk printing `sys.version`
3. An R chunk that loads `tidyverse` and prints `mtcars |> head()`
4. A `sessionInfo()` call
5. The whole thing committed and pushed to a remote you control

. . .

Submit: the URL of your repository. If I cannot clone it and render your file, it is not submitted.

GRADING — pass requires all five. Item 5 is the one that matters and the one students do not anticipate. Clone their repo into a temp folder and render it. If it only works in their own directory, it fails - and say why, because this is exactly how the final project is graded.

Common failures: reticulate missing (item 2), interrupted package install (item 3), absolute file paths or .qmd not committed (item 5).

Before Tuesday

1. Finish the setup verification — do not leave it until Sunday
2. Activate your news subscription
3. Request your Census API key
4. Read **Cohen Chapter 2**, "What are (is?) data?"
5. Start `questions.md` and put at least one thing in it

. . .

Tuesday we write our first real document, and meet the tidyverse.

See you Tuesday

6:15 AM · Bring the questions file